

MASTER DEFENSE & VIVA TECHNICAL MANUAL

CHAMPIONS CLUB

Enterprise Athletic Club & Sports Complex Management Monolith

An exhaustive, end-to-end technical compendium covering Prisma ORM architecture, database schema internals, PostgreSQL GiST exclusion constraints, Clean Architecture execution flows, code walkthroughs, live demo scripts, and examiner defense strategies.

System Monolith: Next.js App Router (16.3.8) + TypeScript Strict + PostgreSQL 18

Persistence: Prisma ORM (7.10.0) + Native SQL Migrations + B-Tree / GiST Indexes

Concurrency Controls: Row Locks (**FOR UPDATE**) + PostgreSQL Exclusion Constraints + Advisory Locks

Background Engine: Transactional Outbox Worker (**SKIP LOCKED**) + Scheduled Reminders

Target Evaluation: Final Project Submission, Code Inspection, Technical Viva & Live Demonstration

1. Executive Architecture & Invariant Rules

Champions Club is built as a **modular Next.js App Router monolith** following Domain-Driven Design (DDD) and Clean Architecture principles. It solves real-world athletic club operations across court reservations, membership subscriptions, retail e-commerce/POS, restaurant dining, staff HR/payroll, and financial double-entry accounting.

Core Engineering Invariants (Examiner Must-Knows)

- **Server-Authoritative Pricing:** Clients never submit prices, discounts, or roles. The server queries the database, applies plan snapshots, and enforces the source of truth.
- **Integer Paise Representation:** All monetary values are 64-bit integers in Indian Paise (₹1.00 = 100 paise). Eliminates IEEE-754 floating-point rounding errors (e.g. $0.1 + 0.2 \neq 0.3$).
- **Asia/Kolkata Business Calendar:** All timestamps persist in UTC (ISO 8601), but operational club days (06:00 to 23:00), daily quotas, and court schedules evaluate strictly in `Asia/Kolkata` timezone.
- **Immutable Financial Snapshots:** Once an invoice is issued, database triggers prevent mutations to line items or totals. Plan benefits and pricing snapshots are frozen in JSON columns at purchase time.
- **High-Concurrency Protection:** Simultaneous race conditions (booking the same court, buying the last racket in stock, claiming the final mixer slot) are defeated at the database engine level using PostgreSQL GiST exclusion constraints and row-level locks (`SELECT FOR UPDATE`).

System Structure Mapping

`src/app/api/*`: Authenticates cookies via Better Auth, validates input via strict Zod schemas.

`src/modules/*`: Isolated business domains (bookings, shop, clubhouse, billing, membership, operations).

`src/worker.ts`: Independent Node daemon claiming outbox jobs via `FOR UPDATE SKIP LOCKED`.

2. Prisma ORM & Schema Deep-Dive

What is Prisma ORM? Prisma is a modern, next-generation TypeScript Object-Relational Mapper (ORM) composed of:

- 1. **Prisma Schema (`schema.prisma`)**: The declarative single source of truth describing models, relations, enums, data types, and database mappings.
- 2. **Prisma Client (`@prisma/client`)**: An auto-generated, type-safe query builder that compiles TypeScript method calls (e.g. `tx.reservation.create()`) into optimized SQL queries.
- 3. **Prisma Migrate**: Declarative migration engine tracking database schema evolution via versioned SQL migration folders in `prisma/migrations` .
- 4. **Prisma Query Engine**: A high-performance binary/Rust-backed query engine that manages connection pools, prepares parameterized SQL statements (preventing SQL injection), and maps relational rows to structured JSON objects.

2.1 The Anatomy of `schema.prisma`

Open `prisma/schema.prisma` . It starts with two configuration blocks:

```
generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma" // Custom generated location for strict isolation
}

datasource db {
  provider = "postgresql" // Target database engine
}
```

2.2 Database Enums

Enum Name	Values	Purpose & Invariant
Role	MEMBER, RECEPTION, CASHIER, KITCHEN, OWNER	Enforces Role-Based Access Control (RBAC). Stored in the User model. Owner has full administrative clearance.
MembershipStatus	ACTIVE, SUPERSEDED	Tracks subscription validity. When renewed or upgraded, older terms transition to SUPERSEDED .
Department	MEMBERSHIP, COURT, SHOP, CLUBHOUSE	Scopes invoices, ledger entries, quotes, and departmental access guards.
ReservationStatus	HOLD, CONFIRMED, CANCELLED, EXPIRED	Life cycle states of court bookings. Only HOLD and CONFIRMED block court availability.
JobStatus	PENDING, RUNNING, DONE, FAILED, CANCELLED	Durable transactional outbox job states. Prevents duplicate execution.

2.3 The "Little Things" in Schema That Examiners Grill On

Prisma Attribute	What it Means Under the Hood	Why It Was Chosen / Examiner Gotcha
<code>@id</code>	Defines the Primary Key constraint (<code>PRIMARY KEY</code> in SQL).	Every database entity requires a unique identifier.
<code>@default(cuid())</code>	Generates a Collision-resistant Unique Identifier (CUID) on insert.	Unlike integer auto-increments (1, 2, 3), CUIDs are unpredictable (prevents enumeration attacks) and safe in distributed environments without central coordination.
<code>@unique</code>	Creates a SQL <code>UNIQUE</code> constraint on a single column.	e.g. <code>email</code> or <code>championsId</code> . Database rejects duplicate inserts instantly with error code <code>P2002</code> .
<code>@@unique([colA, colB])</code>	Composite Unique Constraint spanning two or more columns.	e.g. <code>@@unique([department, originId])</code> prevents an origin order/booking from generating multiple invoices! Or <code>@@unique([userId, key])</code> in Checkout for idempotency.
<code>@@index([colA, colB])</code>	Composite B-Tree Index in PostgreSQL.	Crucial for performance! e.g. <code>@@index([courtId, startsAt])</code> allows PostgreSQL to instantly look up court slots in <code>O(log N)</code> instead of scanning the entire table (<code>O(N)</code>).
<code>@db.Timestampz(3)</code>	PostgreSQL <code>TIMESTAMP WITH TIME ZONE</code> with 3-digit microsecond precision.	Stores explicit UTC offsets. Prevents daylight saving or server clock skew bugs across date calculations.
<code>@db.Date</code>	PostgreSQL <code>DATE</code> type (year-month-day only, without time).	Used for <code>clubDay</code> and <code>dateOfBirth</code> . Prevents timezone shifts from changing a user's birthday or club date.
<code>Json</code>	PostgreSQL <code>JSONB</code> binary JSON storage.	Used for <code>planSnapshot</code> and <code>priceSnapshot</code> . Stores immutable historical snapshots of benefits and discounts at the exact second of purchase.
<code>@relation(..., onDelete: Cascade)</code>	Foreign key cascade deletion constraint.	When a parent <code>User</code> is deleted, its associated <code>Session</code> and <code>Account</code> rows are automatically deleted by the database engine.

2.4 Complete Model-by-Model Breakdown

Group A: Authentication, Users & Business Settings

- **User** : Core customer/staff identity. Contains **email** , **role** , **championsId** (human-readable club ID like **CC-MEM-XXXX**), and relations to all activity.
- **Session** & **Account** : Better Auth session tokens and OAuth/password provider credentials with expiry timestamps.
- **ClubSettings** : Singleton configuration record (**id** : "club"). Holds club operating hours (**openHour** : 6 , **closeHour** : 23), **bookingWindowDays** : 14 , **dailySessionLimit** : 2 , **holdMinutes** : 5 , **cancellationHours** : 12 , **tabLimitPaise** : 500000 (₹5,000), and reminder schedules.

Group B: Memberships & Contactless Access

- **MembershipPlan** : Subscription templates (Gold, Silver, Junior). Configures pricing in paise, duration (e.g. 90 days), discount basis points (**courtDiscountBps** , **shopDiscountBps** , **foodDiscountBps**), and complimentary sessions (**freeSessionsWeek**).
- **Membership** : Realized subscription instance for a user. Holds **startsAt** , **endsAt** , **status** (ACTIVE/SUPERSEDED), and **planSnapshot** (JSON frozen benefits).
- **MemberCard** : Digital member pass holding an opaque encrypted token (**token**) for QR generation and check-in scanning.

Group C: Facility Bookings, Social Mixers & Waitlists

- **Sport** & **Court** : Facility inventory (Tennis, Padel, Badminton, Cricket) with hourly pricing in paise and indoor/outdoor status.
- **Reservation** : Court booking record. Contains **courtId** , **startsAt** , **endsAt** , **clubDay** , **status** (HOLD, CONFIRMED, CANCELLED, EXPIRED), **holdUntil** (5-minute timer), and **pricePaise** .
- **CourtClosure** : Maintenance/operational closure intervals blocking court availability.
- **SocialEvent** : Capacity-gated Friday multiplayer mixers linked to a parent court reservation.
- **SocialParticipant** : Individual player slot in a social event with independent checkout hold and invoice.
- **WaitlistEntry** : FIFO queue entry for fully booked court slots or social events with 30-minute offer windows.

Group D: Retail Pro-Shop & Multi-Channel Inventory

- **Product** : E-commerce catalog item categorized by sport and category.
- **ProductVariant** : Specific purchasable SKU (size, grip, color). Tracks physical **stock** and temporarily **reserved** units.
- **StockMovement** : Audited inventory adjustments (purchase deductions, restocks, damage write-offs) with actor ID and reason.
- **ShopOrder** & **ShopOrderLine** : Counter POS and online delivery orders with delivery address and fulfillment states (PENDING, COLLECTED, DISPATCHED, DELIVERED).

Group E: Clubhouse Dining & Kitchen POS

- **DiningTable** : Restaurant dining table records (Table 1, Table 2, etc.) with seating capacity.
- **KitchenOrder** : Open dining table session tracking **tabEnabled** , **ageConfirmed** (for alcohol), and overall preparation stage. Protected by unique index: only **one active open bill per table!**
- **KitchenTicket** & **KitchenLine** : Granular food orders sent to the kitchen display system (KDS). Tracks revisions and lifecycle (ACCEPTED → COOKING → READY → SERVED).

Group F: Double-Entry Financial Ledger

- **Invoice** & **InvoiceLine** : Server-generated bills across all departments. Snapshotted subtotal, discount, total in paise, and payment due date. Protected against mutation by PostgreSQL triggers.
- **Payment** : Recorded monetary transaction. Holds payment method (LOCAL, CASH, CARD, UPI, GATEWAY), source (LOCAL_SIMULATED, MANUAL_RECORDED, GATEWAY_VERIFIED), and link to **CashShift** .
- **PaymentAllocation** : Links payments to invoices. Enforces that payments cannot exceed invoice totals.
- **Credit** & **Refund** : Credit notes issued for cancellations or returns, and audited cash/gateway refunds.
- **BusinessQuote** : Persisted corporate or event quotes convertible into unpaid business invoices.

Group G: Staff Operations, HR & Payroll

- **CashShift** & **CashPayout** : Cash drawer audit session. Tracks opening float, receipts, refunds, payouts, counted cash, and audited discrepancy explanations.
- **Employee** , **StaffShift** , **LeaveRequest** , **Payslip** : HR management with automated conflict detection and finalized salary snapshots with withholding tax.

Group H: System Reliability & Outbox

- **Job** : Transactional outbox records claimed by background workers.
- **MailMessage** : Queued and sent email notifications with retry tracking and audit redactions.
- **AuditLog** : Immutable operational event log recording actor ID, action, entity ID, and context.

3. Concurrency Protection & SQL Migrations

A common examiner question is: *"Why can't Prisma do everything? Why did you write custom SQL migrations?"*

The Answer: Prisma ORM excels at standard relational mappings and CRUD, but cannot express advanced PostgreSQL engine-level primitives like **GiST Exclusion Constraints**, **Transactional Advisory Locks**, and **Row-level Snapshot Triggers**. These live in the versioned SQL migrations!

3.1 The GiST Exclusion Constraint (Zero Overlap Invariant)

In `prisma/migrations/202610030001_foundation/migration.sql`:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE "Reservation"
ADD CONSTRAINT "court_no_overlap"
EXCLUDE USING gist (
  "courtId" WITH =,
  tstzrange("startsAt", "endsAt", '[') WITH &&
) WHERE (status IN ('HOLD', 'CONFIRMED'));
```

How This Defeats Concurrency Races (Examiner Viva Answer)

If two players click the exact same court and hour slot at the exact same millisecond:

1. Both requests arrive at different Node.js worker threads.
2. Both threads attempt to insert a `Reservation` with status `HOLD`.
3. The PostgreSQL storage engine evaluates the GiST index. It detects that the two time intervals `[startsAt, endsAt)` overlap (`&&`) for the identical `courtId` (`=`).
4. PostgreSQL allows transaction #1 to commit and **instantly aborts transaction #2 with SQL error code 23P01 (Exclusion Violation)**. Double-booking is mathematically impossible!

3.2 Deterministic Row-Level Locking (Deadlock Prevention)

When purchasing multiple inventory items or settling bills, how do we lock rows without deadlocking? Look at `src/modules/shop/service.ts`:

```
async function stockLock(tx: Tx, ids: string[]) {
  // Sort variant IDs deterministically to prevent cyclic lock waits (deadlocks)!
  for (const id of [...new Set(ids)].sort())
    await tx.$queryRaw`SELECT id FROM "ProductVariant" WHERE id=${id} FOR UPDATE`;
}
```

Why Sorting by ID Matters (Deadlock Explanation)

Suppose User A buys Item 1 and Item 2, while User B buys Item 2 and Item 1 simultaneously. Without sorting: User A locks 1 and waits for 2; User B locks 2 and waits for 1.

Deadlock!

With `.sort()`: Both transactions must acquire lock on Item 1 first, then Item 2. One transaction waits cleanly without crashing!

3.3 PostgreSQL Advisory Locks

In `guard_daily_quota()` (migration `202610030002_operations`):

```
PERFORM pg_advisory_xact_lock(hashtextextended(member_id, 0));
```

Locks the member's daily quota counter across concurrent transactions. If a member opens 3 tabs to book 3 courts simultaneously to exceed their 2-session daily limit, the advisory lock serializes the evaluation and rejects the 3rd booking!

4. End-to-End Tracing of Core Interactions

This section details each operational flow: what the user sees, what API is called, what module executes it, the exact Prisma query, and what to say during the demo.

4.1 Feature 1: Booking a Court & Concurrency Protection

- 1 **UI Layer:** `src/app/book/page.tsx` → `src/components/courts-view.tsx`
- 2 **Public API:** GET `/api/public/availability?sport=tennis&date=YYYY-MM-DD`
- 3 **Hold API:** POST `/api/operations/booking` (Idempotency-Key header)
- 4 **Domain Module:** `src/modules/bookings/service.ts` → `holdBooking()`
- 5 **Confirmation API:** PATCH `/api/operations/booking/:id` with action: "confirm"
- 6 **Billing Module:** `src/modules/billing/service.ts` → `payInvoice()`

Under the Hood: Execution & Prisma Code

1. **Slot & Quota Validation:** Checks opening hours (06:00–23:00) and user's daily quota (2 sessions/day). Checks for active maintenance closures (`CourtClosure`).
2. **Benefit & Price Calculation:** Server reads active membership. Gold members get 20% discount; Silver members get 10%. If user has unused complimentary weekly sessions, price is ₹0.
3. **Hold Creation (Prisma Query):**

```
const reservation = await tx.reservation.create({
  data: {
    userId: who.userId,
    courtId: court.id,
    startsAt,
    endsAt,
    clubDay: new Date(input.day),
    status: "HOLD",
    holdUntil: new Date(Date.now() + settings.holdMinutes * 60000), // 5-min timeout
    pricePaise: price.totalPaise,
    priceSnapshot: price.snapshot,
  }
});
const invoice = await issueInvoice(tx, { department: "COURT", originId: reservation.id, ... });
await job(tx, "EXPIRE_BOOKING", "booking-expiry:" + reservation.id, { id: reservation.id }, holdUntil);
```

4. **Immediate Release vs Worker Sweeper:** If user closes modal, client calls `PATCH /api/operations/booking/:id` with `action: "cancel"`, releasing slot immediately. If user disconnects, the durable background worker sweeps expired holds.

Examiner Demo Script for Court Booking

"Examiner, let me demonstrate our race-safe court booking. I am logged in as Aarav (Gold Member). I navigate to Book a Court, select Tennis, and click 07:00 AM on Centre Court. The UI immediately opens a checkout modal. On the server, a 5-minute cryptographic HOLD reservation and a snapshotted invoice in integer paise have been generated inside a single ACID transaction. Notice the 20% Gold discount automatically applied by the server."

"If another member tries to click this same slot right now, PostgreSQL's exclusion constraint and hold check will reject them with 'Someone has already reserved this session'. When I click Confirm, the system executes `payInvoice()`, marks the reservation `CONFIRMED`, and issues an immutable receipt."

4.2 Feature 2: Buying Stuff (Pro-Shop Retail E-Commerce & Counter POS)

- 1 **UI Layer:** src/app/shop/page.tsx → src/components/shop-catalog.tsx
- 2 **Cart Storage:** Scoped by user ID in LocalStorage (prevents guest cart leak into member accounts).
- 3 **Checkout API:** POST /api/operations/order with items, delivery flag, address.
- 4 **Domain Module:** src/modules/shop/service.ts → createOrder()
- 5 **Fulfillment API:** PATCH /api/operations/order/:id (actions: collect, dispatch, return)

Under the Hood: Execution & Prisma Code

1. **Deterministic Stock Lock:** Locks all product variant rows in ascending ID order:

```
await stockLock(tx, input.items.map(i => i.variantId));
for (const item of input.items) {
  const variant = await tx.productVariant.findUniqueOrThrow({ where: { id: item.variantId } });
  assert(variant.stock - variant.reserved >= item.quantity, "OUT_OF_STOCK", "Insufficient stock.");
  await tx.productVariant.update({
    where: { id: item.variantId },
    data: { reserved: { increment: item.quantity } } // Atomically reserve stock
  });
}
```

2. **Fulfillment & Restock:** When cashier clicks "Collect" (pickup) or "Dispatch" (delivery), **reserved** is decremented and physical **stock** is permanently decremented. If an item is returned, the cashier specifies whether to restock or scrap damaged goods, generating a **StockMovement** audit row.

Examiner Demo Script for Retail POS

"Examiner, let's explore our multi-channel retail system. The customer portal and the cashier counter share the exact same synchronized inventory pool. When an order is placed, stock is not blindly decremented—it transitions into a **reserved** state with a 5-minute expiry hold. This prevents phantom stock depletion if a customer abandons checkout."

"Now in the Staff Desk under Inventory, the cashier sees the pickup order, inspects the line items, and clicks 'Mark Collected'. Stock is permanently settled and an audited stock movement record is logged."

4.3 Feature 3: Memberships, Junior Age-Checks & Digital QR Pass

- 1 **UI Layer:** src/app/memberships/page.tsx & src/app/account/page.tsx
- 2 **Purchase API:** POST /api/me/membership with plan ID, action (purchase/renew)
- 3 **QR Card API:** GET /api/me/card → Generates dynamic QR token via qrcode package.
- 4 **Domain Module:** src/modules/membership/service.ts → purchaseMembership()

Under the Hood: Execution & Prisma Code

1. **Junior Plan Age Guard:** If plan is **juniorOnly**, server checks the user's birthdate:

```
const age = ageAt(user.dateOfBirth, termStartsAt);
assert(age < 18, "JUNIOR_RESTRICTION", "Junior plans require the applicant to be under 18.");
```

2. **Contiguous Term Extension:** If Aarav has 10 days remaining on his Gold Plan and renews, the new term starts **on the exact second the current term expires** (**endsAt**), preserving his remaining days!
3. **Digital Pass Security:** The Member QR does not contain the database primary key. It contains an opaque crypto token (**token** in **MemberCard**). If compromised, the user can click "Revoke & Reissue" without deleting their account.

5. The Staff Desk Workspaces (Step-by-Step)

The Staff Desk (`/staff`) is the operational hub for Club Receptionists, Cashiers, Kitchen Staff, and Club Owners. Each workspace implements role-segregated UI and backend guards.

5.1 Workspace 1: Reception (Calendar, QR Check-In & Maintenance)

Feature / Interaction	Backend API & Service Function	What Happens Under the Hood
Day Calendar Grid	<code>GET /api/operations/reception?day=YYYY-MM-DD</code>	Loads hourly reservation matrix across all 4 sports. Filters closures and highlights confirmed slots vs holds.
Single-Use QR Check-In	<code>PATCH /api/operations/booking/:id action: "checkin"</code>	Receptionist scans QR or searches Champions ID. Server validates that current time is within the 30-minute arrival window before session start. Sets <code>checkedInAt = NOW()</code> . Subsequent scans are rejected as duplicates.
Court Maintenance Closure	<code>POST /api/operations/closure</code>	Schedules emergency or planned closure. PostgreSQL trigger <code>guard_court_calendar()</code> checks if active bookings exist on that interval and blocks closure until affected bookings are cancelled with reason.
CRM Enquiries & Quotes	<code>GET /api/operations/crm</code> <code>PATCH /api/operations/crm/:id</code>	Reception manages incoming public leads, assigns staff, logs activity notes, generates membership quotes, and converts leads to registered accounts.

5.2 Workspace 2: Cashier POS & Clubhouse Dining

Feature / Interaction	Backend API & Service Function	What Happens Under the Hood
Opening Cash Float	<code>POST /api/staff/admin action: "cashOpen"</code>	Cashier counts drawer cash and logs opening float (e.g. ₹2,000 = 200,000 paise). The system locks the user to exactly one open cash shift . Cash cannot be accepted until a shift is open!
Open Table Dining Order	<code>POST /api/operations/bill</code>	Cashier selects table (e.g. Table 4), adds menu items (drinks, snacks), and enters member or guest ID. Enforces unique index: only one open bill per table. Alcohol requires verified 21+ birthdate or cashier override.
Kitchen Tickets (KDS)	<code>POST /api/operations/kitchen</code>	Items generate a <code>KitchenTicket</code> with revision 1. Kitchen sees ticket on live display and transitions state: <code>ACCEPTED → COOKING → READY</code> . Cashier marks <code>SERVED</code> . Uses optimistic locking (<code>version</code>) to reject stale updates.
Member Tabs & Overdue Guard	<code>PATCH /api/operations/bill/:id action: "tab"</code>	Charges dining bill to member's account. Checks that outstanding tab does not exceed <code>tabLimitPaise</code> (₹5,000) and that no tab charges are overdue beyond <code>tabDueDays</code> (7 days).
Closing Shift & Float Audit	<code>POST /api/staff/admin action: "cashClose"</code>	Cashier enters counted physical cash. Server calculates: <code>Expected = OpeningFloat + Receipts - Refunds - Payouts</code> . If counted <code>!=</code> expected, cashier must submit an audited explanation before the shift closes!

5.3 Workspace 3: Owner Dashboard, HR Payroll & Financial Reports

Feature / Interaction	Backend API & Service Function	What Happens Under the Hood
Executive Financial Ledger	<code>GET /api/staff/reports?preset=month</code>	Double-entry accounting summary. Computes billed revenue, collected payments, credit notes, and outstanding debt broken down by department and payment method. Clicking any figure drills down into supporting invoices!
CSV Formula Injection Guard	<code>GET /api/staff/payroll-export</code>	Sanitizes CSV cell values. Prepends single quote (<code>'</code>) to cells starting with <code>=, +, -, @</code> to prevent Excel formula injection attacks when opened in spreadsheets.
Staff Shift Scheduling	<code>POST /api/staff/admin action: "shift"</code>	Owner assigns work shifts to employees. PostgreSQL GiST exclusion constraint (<code>shift_no_overlap</code>) and trigger prevent scheduling overlapping shifts or shifts during approved leave.
Leave Approvals & Payslips	<code>POST /api/staff/admin action: "payslip"</code>	Finalizes monthly payslips. Computes base salary, applies configured withholding tax basis points (<code>payrollTaxBps</code>), and stores an immutable JSON snapshot. Staff can view only their own payslips.

6. The Durable Background Worker & Transactional Outbox

The application runs an independent background daemon: `src/worker.ts`. It operates decoupled from Next.js web requests to guarantee that asynchronous jobs execute reliably even if web traffic spikes or server restarts occur.

6.1 The Transactional Outbox Pattern

When a court hold is created or a password reset is requested, the system does not immediately call an external email service or fire an untracked `setTimeout`. Doing so risks the **dual-write failure problem** (e.g. database commits, but email fails or Node process crashes).

Instead, the business service inserts a record into the `Job` table **inside the exact same ACID database transaction!**

6.2 Job Claiming via `SKIP LOCKED`

Look at `src/modules/mail/worker.ts`:

```
export async function processNextJob() {
  const jobs = await db.$queryRaw<Job[]>`
    UPDATE "Job" SET status='RUNNING', "lockedAt"=NOW(), attempts=attempts+1
    WHERE id = (
      SELECT id FROM "Job"
      WHERE (status='PENDING' AND "runAt" ≤ NOW())
            OR (status='RUNNING' AND "lockedAt" < NOW() - INTERVAL '5 minutes')
      ORDER BY "runAt"
      FOR UPDATE SKIP LOCKED
      LIMIT 1
    )
  `;
  RETURNING *`;
  ...
}
```

Why This Is Industry-Standard Architecture

1. `FOR UPDATE SKIP LOCKED` :

If you run 5 worker processes across a cluster, each worker grabs a pending job and instantly locks it. Other workers skip the locked row and grab the next job without blocking! No worker ever processes the same job twice.

2. **Zombie Job Recovery:**

If a worker process crashes mid-job, the row remains in `RUNNING` status. The clause `"lockedAt" < NOW() - INTERVAL '5 minutes'` allows another worker to reclaim and complete the orphaned job!

3. **Scheduled Reminders:**

The worker automatically synchronizes 7-day, 1-day, and 0-day membership expiration reminders every 60 seconds without burdening web requests.

7. Top 15 Killer Viva Questions & High-Score Answers

Memorize these responses. These are the exact high-difficulty questions examiners love to ask to distinguish top-tier engineers.

Q1: Why did you store money in integer paise instead of float or Decimal?

Answer: "In IEEE-754 floating-point arithmetic used by JavaScript, binary fractions cannot represent decimal fractions like 0.1 or 0.2 accurately. For instance, $0.1 + 0.2 \equiv 0.30000000000000004$. In financial software, accumulating fractional rounding errors over thousands of ledger entries causes balance sheet discrepancies and auditing failures. By storing money as integer paise (1 INR = 100 paise), every addition, discount, and credit is exact to the integer."

Q2: What is the purpose of the Idempotency-Key header on mutations?

Answer: "If a customer clicks 'Pay' and their network drops before receiving the response, their mobile app or browser will automatically retry. Without idempotency, the server would charge their card or create an invoice twice. With our idempotency guard, the server hashes the actor ID, idempotency key, and request payload into the `Checkout` table. If an identical request arrives, the server returns the cached result with `replayed: true` without repeating the financial charge."

Q3: What happens if two members click the same court slot at the exact same millisecond?

Answer: "We defend against this at two layers. First, at the application layer, we acquire an exclusive row lock on the court via `courtLock()`. Second and most importantly, at the database engine level, we enforce a PostgreSQL GiST exclusion constraint (`EXCLUDE USING gist`) using the `btree_gist` extension. The database evaluates time interval overlaps in `[startsAt, endsAt]` for that court ID and rejects the colliding transaction with SQL error code `23P01`. Exactly one succeeds; the second receives an explicit slot-taken error."

Q4: Why do you store plan benefits as a JSON snapshot in the Membership table instead of just referencing planId?

Answer: "Membership plans evolve over time—the club owner might increase the price of the Gold Plan from ₹5,000 to ₹6,000 or lower court discounts from 20% to 15%. If we only referenced `planId`, an active member who paid ₹5,000 under the old contract would suddenly lose their promised benefits or have inaccurate historical invoices. By storing an immutable `planSnapshot` JSON object on the `Membership` row, the customer's agreed benefits remain frozen for their entire term."

Q5: Explain the difference between Invoice, Payment, and PaymentAllocation. Why three models?

Answer: "This follows standard double-entry accounting. An **Invoice** represents a debt or obligation to pay (billed revenue). A **Payment** represents real money received from a customer (cash, card, UPI, gateway). A **PaymentAllocation** represents the mapping between payment and debt. This separation enables real-world scenarios like partial payments (paying ₹1,000 toward a ₹3,000 bill), bulk payments (one payment settling three court bookings), and credit notes without corrupting past transaction history."

Q6: How does the Cash Shift Float Reconciliation prevent cashier theft?

Answer: "A cashier cannot accept cash without opening a shift and declaring their opening drawer float. Throughout the shift, every cash payment, cash refund, and cash payout automatically links to that cashier's active `cashShiftId` under a row lock. At closing, the cashier inputs physical counted cash. The system compares counted cash against calculated cash (`Opening + Receipts - Refunds - Payouts`). If there is any discrepancy, the system refuses to close the shift until the cashier provides an audited written reason."

Q7: How does your background worker avoid race conditions when multiple worker processes are running?

Answer: *"We use the PostgreSQL query clause `FOR UPDATE SKIP LOCKED LIMIT 1`. When Worker A queries for a pending job, PostgreSQL locks that specific row and returns it. If Worker B queries concurrently, PostgreSQL skips the locked row and returns the next pending job. This provides lock-free, zero-conflict horizontal scalability across multiple background workers."*

Q8: Why did you choose a Modular Monolith instead of Microservices?

Answer: *"In an athletic sports club, consistency is paramount—a court booking must atomically create an invoice, reserve inventory, check member quotas, and enqueue notification jobs. In a microservices architecture, this requires complex distributed two-phase commits (2PC) or Sagas prone to network failure and partial state inconsistency. A modular monolith allows us to maintain strict module boundaries while executing critical multi-domain workflows inside single, lightning-fast PostgreSQL ACID transactions."*

8. Live Demonstration Cheat Sheet

Step & Scenario	Login Credentials	Exact Actions to Demonstrate
1. Discover & Enquire	Public Visitor (No login)	1. Open homepage (http://localhost:3000). 2. Scroll to photographic sports selector (Tennis, Padel, Badminton, Cricket). 3. Fill out public enquiry form (e.g. name, email, sport, inquiry message). 4. Submit and verify success toast.
2. CRM & Lead Intake	reception@champions.local Password: Champions2026!	1. Login as Receptionist. 2. Navigate to Staff Desk → CRM & Enquiries tab. 3. Locate the new lead submitted in Step 1. 4. Assign lead to staff, add an activity note, and generate a membership quote.
3. Member Court Booking	member@champions.local Password: Champions2026!	1. Login as Aarav (Gold Member). 2. Navigate to Book a Court → Tennis → Select tomorrow 08:00 AM. 3. Show 5-minute hold timer and server-applied 20% Gold Plan discount. 4. Click "Confirm Local Payment" → Show invoice and confirmed booking. 5. Open My Account → Show active QR Member Pass and itemized receipt.
4. Reception Check-In	reception@champions.local Password: Champions2026!	1. Switch to Reception session. 2. Go to Reception & Calendar tab. 3. Search Aarav's booking or scan QR code. 4. Execute Single-Use Check-In. Show arrival timestamp recorded and button disabled.
5. Retail Pro-Shop POS	cashier@champions.local Password: Champions2026!	1. Member visits Shop → Adds Tennis Racket to cart → Chooses Pickup → Checks out. 2. Cashier opens Staff Desk → Inventory tab. 3. Demonstrates reserved stock decrement. 4. Marks item as "Collected" → Demonstrates physical stock reduction and audit entry.
6. Dining POS & Kitchen Display	cashier@champions.local & kitchen@champions.local	1. Cashier opens POS Dining → Selects Table 3 → Adds Clubhouse Sandwich & Fresh Lime. 2. Kitchen opens Kitchen KDS → Sees incoming ticket → Clicks "Accept" → "Cooking" → "Ready". 3. Cashier marks items "Served". 4. Settles bill via Member Tab → Closes table.
7. Cash Reconciliation & Owner Audit	owner@champions.local Password: Champions2026!	1. Owner opens Reports & Revenue. 2. Filter by "This Month". Click on Court Booking revenue → Drills down into every supporting invoice. 3. Inspect Cash Shifts tab → Shows audited opening floats and zero discrepancies. 4. Open People & Payroll → Show Neha's finalized payslip and print-ready tax withholding snapshot.

End of Comprehensive Submission & Viva Technical Guide

Prepared for Champions Club System Evaluation · All Invariants & Tests Verified against PostgreSQL 18